

Q1. [4] Trace the output of this code: arr [2] = {100,200} for (i = 0; i<2; i++) { arr[i] + = 20 fork() arr[i] + = 20 } print(arr)	Q2. [4] You are a famous gaming content creator. You are playing the game, using OBS to capture Screen and using a tracker app to track all your statistics. So these processes are running at the same time To give you a more detailed Idea, You are gaming in 1080p resolution where each frame is approximately 6MB and you are running in 60 Frame Per Second, thus making each second worth of footage = 360MB/S. Given this context, which IPC protocol is needed to achieve this setup. [Give two points that will benefit your preferred protocol.]
Q3. True/ false [2] a. Once a process is created the total size is allocated for it by OS and Its fixed for the rest of its lifespan	b. A faulty OS somehow deleted the PCB block and the actual process memory layout information is still in the memory. OS can still Run the process properly after the event,

Q1. [4] Trace the output of this code: arr [2] = {100,200} for (i = 0; i<2; i++) { If (fork(>)>0) {arr[i] + = 20} else {arr[i] + = 30} } print(arr)	Q2. [4] You are making a bank management system where hundreds of transactions are being done on your server. There is one admin process that updates the database and checks the validity of the transaction and other processes are transaction processes which send the transaction info. Given the context, which IPC protocol will be suitable for this scenario. Give 2 pointers on why mentioned protocol is a better choice
Q3. True/ false [2] a. Cascading termination can solve the zombie process issues.	b. In one line, describe what OS does to an orphan process.

Q1 True/False → [2]	a. A process can only be in a running state if its prior state was ready state b. In IPC, shared memory requires more kernel level intervention than Message Passing Queue
Q3. [5] Trace the output of this code: a= fork() If (a >0) { b = fork()	Q2. [3] You have two process, one is IO bound meaning it does small portion of its execution in CPU (lets say on average 0.5 second each burst) and mostly spends time in IO operations. Another is CPU bound process which spends most of its time in CPU execution (lets say 100 seconds

<pre>If (b>0) { wait() print("p2")} else {wait() print("not p2")} } print("end") wait() If (a==0) {print ("end 2")}</pre>	each burst). If your OS context switches after a fixed time of lets say 1 second, explain in 2 points, how the first process and second processes performance will be impacted.
--	--